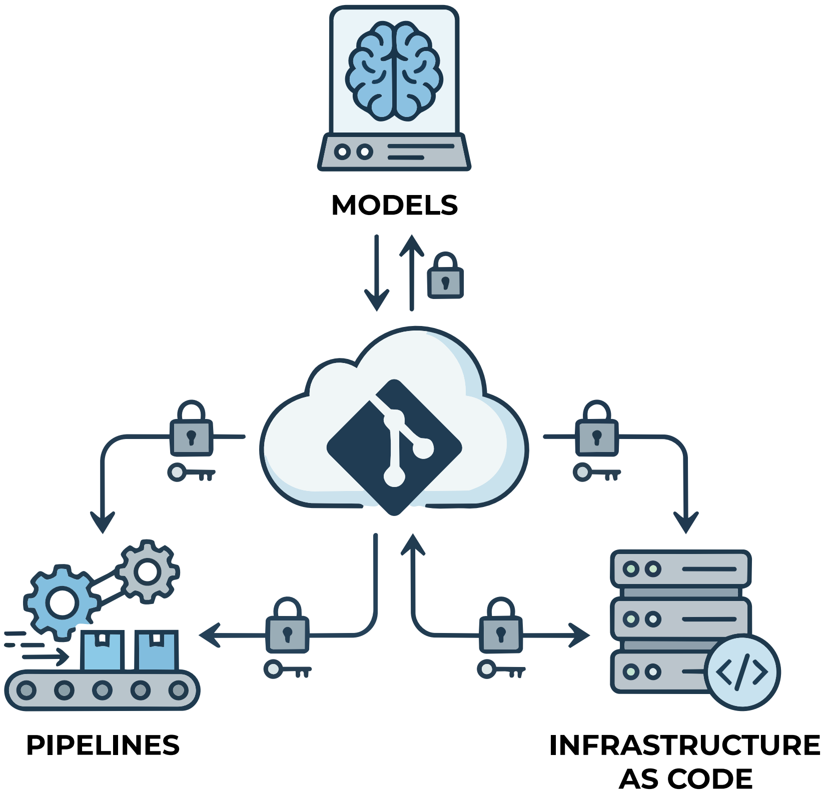
GitOps: Addressing the Challenges of AI Workloads

GitOps helps to deal with the complexity of modern AI workloads, with Git serving as the single source of truth for models, pipelines, and infrastructure configurations.



Artificial intelligence and machine learning applications are vital for today's business operations. Nevertheless, deploying an AI model is a complicated process due to many connected components, such as training code, datasets, model files, data processing pipelines, deployment configurations, and computing infrastructures. The human-centred management of these components can be fraught with configuration mistakes, absence of consistency in environments, and difficulties in getting results.

GitOps introduces a new approach to this process whereby Git functions as the main source of truth for application code, infrastructure configurations, and deployment instructions. After an approved change is submitted to Git, the changes are verified and the required target environment is updated automatically. Therefore, every modification is traceable and reversible.

With Infrastructure as Code, continuous integration, and continuous delivery, the use of GitOps ensures the automated training, testing, deployment, and monitoring of machine learning models and software used in production.

GitOps fundamentals

GitOps is a procedure to deploy applications and manage infrastructure using Git repositories. Git acts as the main reference document. The expected state of the system, application, and infrastructure is included in the version-controlled system files. The files can include descriptions about containers, computing resources, networking specifications, storage requirements, and application versions.

GitOps consists of four main principles. First, the entire system is defined in a declarative way. This means that the configuration files define the desired state, rather than listing the steps needed for the setup. Second, any approved change is kept in the Git repository. This enables creating a full record of what changes have been made, when, and why. Third, automated systems implement the approved amendments. Finally, software programs check the actual state of the system in comparison to the state registered in Git continuously.

The process of checking is known as reconciliation. The GitOps controller reconciles the current state against the approved state and restores the original version if the two do not match.

A typical GitOps workflow starts when a developer updates a configuration file and creates a pull request. Team members review and approve the proposed change before it is merged. An automated controller, such as Argo CD or Flux, then detects the updated repository state and applies it to the Kubernetes environment. If a deployment fails, the team can roll back to a previously stable version stored in Git. Therefore, GitOps provides automation, traceability, consistency and